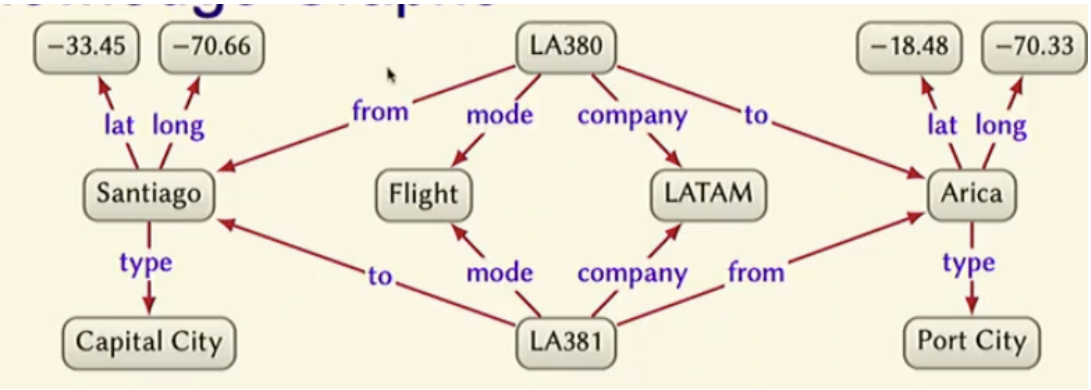
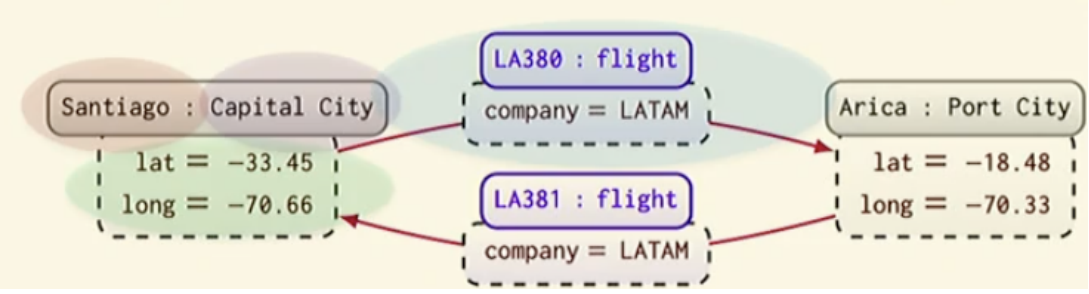
- ci concentreremo nei grafi della conoscenza, che è un modo di rappresentare la conoscenza
- Cose che abbiamo fatto in fondamenti, le inferenze
  - Abbiamo proposizioni/enunciati, cioè le premesse, cose vere
  - Le inferenze lavorano solo sulle premesse
    - Logica propozionali
      - Abbiamo enunciati, o veri o falsi
      - E delle tavole di verità
    - Induzioni
      - Se noi sappiamo qualcosa, allora possiamo dedurre qualcos'altro
    - Abduzione
    - Logiche del primo ordine
- Ci sono 2 approcci per catturare pattern di problemi
  - Problema risolutivo dichiarativo
    - La soluzione è fatta attraverso inferenze nel linguaggio
    - Es. Prolog
      - Si risolvi in maniera dichiarativa
    - Supponiamo che noi siamo online, però non riusciamo a raggiungere il nodo E, facciamo un sistema di diagnostica
      - Un sistema: esplora il grafo, ogni nodo, e controlla qual'è il nodo in mezzo che è offline
      - ```
        reaches(X,X)  :-  node(X), not offline(X).
        reaches(X,Z)  :-  reaches(X,Y), connected(Y,Z),
                          not offline(Z).
        ```
        - Notiamo che usiamo il predicato not, quindi facciamo inferenze in tanti modi
      - Gli output vengono calcolati attraverso le inferenze
  - Sistemi di conoscenza (knowledge bases, KB)
    - E' un sistema che persiste un insieme di enunciati, da accesso ad essi e da conclusioni da essi
    - E il nostro obiettivo e manipolare la conoscenza, e la manipoliamo con
      - Fatti + conoscenze generiche
      - Ragionamento: inferenze e inferenze dalle query
    - La differenza tra una base di conoscenza e database: nel database non fai inferenza
        ```

    - Mentre la similarità, è che nelle KB puoi fare create, read, update, delete
    - Hai informazioni persistenti e localizzate

      1 tipologia di sistema di conoscenza sono i **grafi di relazioni**
- Sono grafi con relazioni binarie
- Sono utili siccome è efficiente processare dei grafi
    - E possono aggregare dati delle varie entità
    - Ed in più è più facile fare query rispetto a sql
- E sono facili da applicare e creare
- Definizione matematica: è una quintupla
    - E: set di simboli per identificare le entità (aka gli uuid)
    - L: set di literali (il valore di certe entità, le label)
    - T: il tipo dei simboli
    - P: un insieme di simboli per identificare le relazioni
    - A: insieme di assiomi, che si dividono in
        - Assiomi per associare entità e literali
            - Essendo un grafo, se noi vogliamo dire "piano elettrico", quella è un entità
            - E poi dire tipi di piani elttrici, quelli sono literali
        - Assiomi per fare unire varie entità
- Abbiamo 2 approcci per aggiungere/rimuovere/modificare
    - Modello a grafo con archi diretti ed etichettati

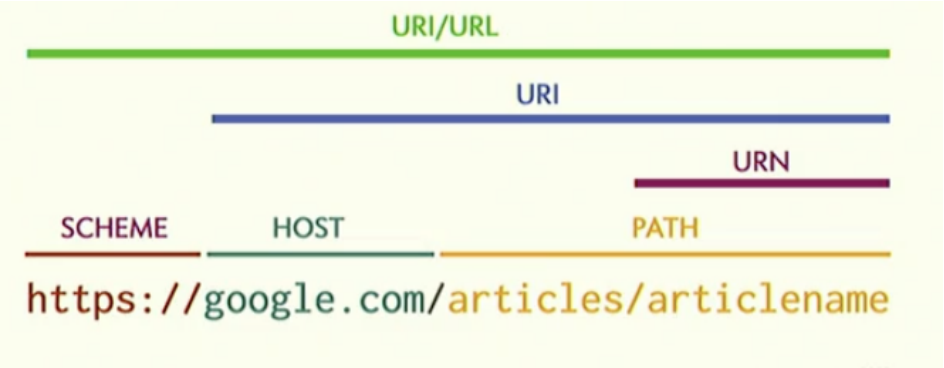


        - Qui abbiamo solo entità e relazioni
    - Modello a proprietà



        - Abbiamo concetto di entità, proprietà, tipo, nodi e relazioni ecc

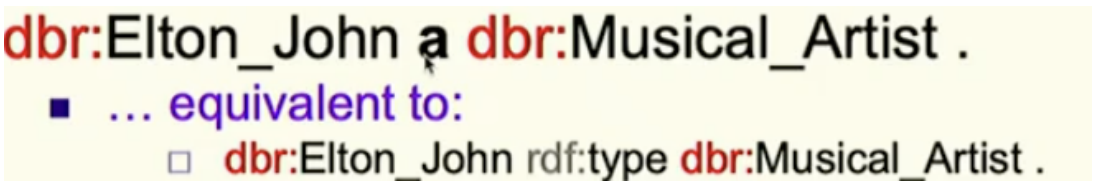
- Un altro sistema di grafo delle conoscenze è RDF
  - Definizione: è un datamodel per condividere e pubblicare dati sul web
    - Basato su triple con iri, literali e blank nodes
    - I vocabolari servono per organizzare le informazioni
  - E' un grado diretto etichettato con un insieme di nodi, labels e tuple
    - E le nostre triple sono un sottoinsieme delle tuple, labels e nodi
  - E' un modello di dati che si basa su 3 principi
    - Triple, unità basi per organizzare le informazioni <Entità, tipo, valore>
      - E' possibile avere le quadruple, quando lo si fa è per dare contesto/metatesto
    - Le triple sono organizzate in grafi diretti etichettate <soggetto, predicato, oggetto>
      - Aka gli archi
    - Uri
      - E' una stringa di caratteri che identifica univoca una risorsa su internet
      - Es.
        - URL, che è difiso in schema - host - path



  - Ogni rdf usa dei prefissi per semplificare, es. dbr: ciao | <dbr:si> è come dire <ciao:si>
    - Ci permette di risparmiare memoria, duh



    - rdf:type è per dire "e", e può essere lo stesso di dire a



- ecc eecc
- Ed ogni RDF può avere 3 tipologia di nodi
  - IRI (soggetto, predicato, oggetto)
    - Vincoli: il predicato deve essere un iri
    - Il soggetto può essere o un iri o un blank node
    - L'oggetto può essere
      - iri
      - blank node
        - Si identifica con o _:name oppure []
        - Non ha un identificativo globale
      - literato
        - Stringhe, numeri, dati
          - Stringhe inizia e finisce con "
          - Il tipo della lingua con @ (@en @it)
          - ^^ intero
        - Sono praticamente i dati
        - Si specifica il tipo per questioni di semantica
  - Blank nodes
  - Literal
- Ed esistono i vocabolari, che sono praticamente standard accettati
  - Sono social agreement, così dall'esterno le persone sanno già come girarsi
  - Sono consigli per developer, così facendo è più facile organizzarsi tra sviluppatori di diverse aziende
  - Es. specifica doc: solo i documenti
- E' un vocabolario basato sulla agreement sociale
  - rdf:scheme è un esempio
- Siccome sono dati molto caotici, bisogna usare proprietà linguistiche siccome non c'è un pattern universale
  - Piace tanto alle llm
    Però può essere **ambiguo**
    - Se io cercho "slayer" di bande ne abbiamo milioni
    - Per questo che diamo gli identificativi
  - Tra varie basi di conoscenze, è possibile che siano modellate in maniera diversa
    - Su wikipidia abbiamo membro di: A B
      - E' data first, più dati ma non organizzati
    - Su dbpidia potremmo avere membro di: A && fa parte di: B

- E' model first, il vocabolario è organizzato
  https://elearning.unimib.it/mod/kalvidres/view.php?id=1508548
  33:00